

Post-Disaster Rescue Guidance via VLM Fine-Tuning

CSE499A – Section 15, Group 5, Update 3

Name	Student ID
Abdullah Al Noman	2022095042
Tamanna Akter Mou	2211951042
Aryan Sami	2231407042
Ridita Afrin Riya	2211622042
Abrar Mohammed Tanzim Alam	2222864042

Work Done by Abrar Mohammed Tanzim Alam

My primary objective this week was to reproduce the fine-tuning and evaluation pipeline of the DisasterM3 (DM3) paper (Wang et al., NeurIPS 2025, arXiv:2505.21089v2), whose released repository (Junjue-Wang/DisasterM3) provides only benchmarking code—no training scripts. This required self-assembling the full training, merging, and evaluation infrastructure from scratch on Kaggle’s single-T4 hardware, matching the hyperparameters in Appendix B.3 (LoRA rank=64, $\alpha=16$, dropout=0.05, AdamW lr= 2×10^{-4} , cosine schedule, global batch 256, 1 epoch, vision encoder frozen). The implementation spans six sequential notebooks:

1. **Dataset Inspection** (disasterm3_dataset_inspection.ipynb): Audited the 87 MB `train_release.json` manifest (HF mirror Kingdrone-Junjue/DisasterM3). Found 37,204 Referring Expression Segmentation entries (40% of 92,968 total) had empty `training.answer` fields (ground truth is a mask path, not text)—invalid for text generation. Curated effective set: **55,764 valid instruction pairs** across 8 tasks (BDC 14,531; DBBR 7,766; Caption 7,766; Advice 7,765; RDC 7,337; DSR 7,090; Relational 1,882; DTR 1,627).
2. **Dataset Mirroring** (disasterm3_dataset_mirror.ipynb): Transferred the full 44 GB DM3 dataset (26,988 bi-temporal satellite images, 36 disaster events) from HF to a permanent private Kaggle Dataset, staging through `/tmp` (~70 GB) to bypass the 20 GB `/kaggle/working` cap—eliminating repeated multi-hour downloads across sessions.
3. **QLoRA Fine-Tuning** (train_qwen_disasterm3.ipynb): Built the training pipeline (`transformers+peft+trl` SFT-Trainer) for a single T4 (16 GB VRAM), replacing the paper’s $4 \times H100$ full-precision setup with 4-bit NF4 QLoRA, fp16 (T4 lacks native bf16), SDPA attention, and grad-accum 256 to preserve the paper’s global batch size. Fixed a LoRA misconfig where the vision tower wrongly received adapters (190.4M vs. expected 161.5M trainable params) via a regex-anchored `target_modules` fix restricting adapters to `model.layers.*`. Ran 5 checkpoint-resume sessions (12-hr limit each), reaching step 102/217, loss 0.83→0.54; backed up `trainer.state.json` and adapter weights to Kaggle Dataset + HF Hub (AbrarAlam/disasterm3-qwen25v17b-qlora).
4. **LoRA Merge** (1_merge_and_push_model.ipynb): Merged the QLoRA adapter with base Qwen2.5-VL-7B in FP16, pushed the fused 15 GB checkpoint to HF (AbrarAlam/disasterm3-qwen2.5v17b-mergedFP). Required an isolated session to avoid `numpy/scipy` ABI mismatches and `triton` crashes from conflicting `vllm` requirements.
5. **Benchmark Mirroring** (2_dataset_creation.ipynb): Downloaded all 22,381 benchmark assets (test images, segmentation masks, `benchmark_release.json`) from HF’s `DisasterM3_Bench` folder (11 GB), packaged into a permanent Kaggle Dataset (disasterm3-bench-mirror, 13.7 GB zipped). Overcame authenticated rate-limiting and `snapshot.download` deadlocks via token-injection and cache-bypass fallback.
6. **vLLM Evaluation** (3_vllm_evaluation.ipynb): Configured the paper’s official eval script (`pyscripts/run_vllm.py`) with vLLM serving to benchmark the merged model on the 6 MCQ tasks from Table 2 (DSR, DTR, BBR, BDC, DRE, ORR). Open-ended tasks (Caption, Restoration Advice) excluded (GPT-4.1-as-judge cost ~\$300, over capstone budget).

Work Done by Tamanna Akter Mou

1. **Proposed-Solution-Report**: After observing the implementation progress of Abrar, Noman, Aryan, and Ridita, I organized the project narrative into the proposed-solution report I connected each member’s implementation to the final two-tier framework so the report does not look like separate unrelated tasks.
2. **Orchestration and Evaluation Writing**: I expanded the command-center side of the paper by describing practical outputs such as damage reports, social media alerts, medical staging, HR allocation, public advisories, and reconstruction plans. I also emphasized rescue-actionability evaluation so the generated text is judged by safety and usefulness, not only by text similarity. This helped align the update paper with the proposed-solution report and made the weekly progress easier to defend during presentation.

Work Done by Abdullah Al Noman

1. **Super-Resolution Implementation**: Implemented the super-resolution model (from last week’s presented paper, *Shakya-From Pixels to Semantics*) to enhance visual fidelity and structural detail of post-disaster AIDER dataset.
2. **Data Engineering and Tensor Construction**: Built `prep_vrt_input.py` to satisfy the Video Restoration Transformer’s (VRT) spatial-temporal tensor requirement, since a single static image (AIDAR_collapsed_building_image0464.jpg) can’t be directly ingested by a model that extracts dependencies across a temporal window. The script created a REDS-standard nested directory (`/testsets/custom_disaster/LQ/000`) and replicated the static 1024×1024 frame four times (0000.png–0003.png), emulating a zero-motion pseudo-video clip to produce the required 5D tensor $[B, T, C, H, W] = [1, 4, 3, 1024, 1024]$.
3. **Weight Configuration and Mapping**: Downloaded The pre-trained mathematical weight configurations matching the `001_VRT_videosr_bi_REDS_6frames`. Then mapped The weights (.pth checkpoint file) to the codebase’s strict local pathway (`/model_zoo/vrt/`) to ensure proper structural state-dict binding.
4. **Inference and Memory Optimization**: The massive computational overhead of generating a 4096×4096 output array was mitigated by partitioning the 1024px input space into small, sequential spatial tiles of 256×256 pixels. The runtime command locked the temporal processing slice to 4 frames (`--tile 40 256 256`) and implemented a strict 20-

pixel overlapping boundary padding (`--tile_overlap 2 20 20`) to insulate the data loader from threading instability while managing the free-tier GPU constraints.

5. **Execution and Output Generation:** Finally, The overlapping tile margins were seamlessly fused together using a recurrent blending utility to eliminate block artifacts or visible grid seams across the upscaled geography. This pipeline cleanly bypassed the repository’s native dataset parsing limitations, generating the final 4X super-resolved structural assessment asset directly to the disk at `/results/.../000/0000.png` with a crisp, verified resolution of exactly 4096×4096 pixels.

Work Done by Aryan Sami

1. **Resolution and Localization Gaps Identified:** Documented how 10m ground sampling distance limits fine-grained damage detection in dense urban settings (e.g., Mexico earthquake failure case), with building localization—not damage classification—remaining the weakest pipeline stage, constraining how reliably free satellite data can substitute for VHR imagery.
2. **Multi-Source Data Integration Rationale:** Outlined how combining pre-/post-event optical (Sentinel-2) and radar (Sentinel-1) imagery improves robustness to cloud cover and lighting versus single-source approaches, directly supporting more reliable, damage-aware rescue guidance.
3. **Dataset-to-Model Mapping:** Mapped the xBD-S12 dataset (spatiotemporally aligned Sentinel-1/2 patches paired with corrected xBD building-damage labels) against our rescue-oriented instruction-response format, clarifying how satellite comparisons and damage classifications convert into training-ready multimodal examples.
4. **Area of Interest and Acquisition Setup:** Defined the fire-affected extent for the Jan 2025 Palisades wildfire via its perimeter boundary; configured acquisition for Sentinel-1 (VV/VH, orbit 137, via Google Earth Engine) and Sentinel-2 L2A (tile 11SLT, via phidown), pulling matched pre/post scenes (S1: 2024-11-27 / 2025-02-07; S2: 2024-11-28 / 2025-02-01).
5. **Multi-Sensor Co-Registration:** Reprojected all four rasters (S1 pre/post, S2 pre/post) onto a shared 4m grid for pixel-level alignment across sensors and time steps—operationalizing the multi-source rationale above by making optical and radar streams directly comparable, not merely co-located.
6. **Model Inference Execution:** Ran the pretrained two-step ensemble (3 seeds, covering building localization + damage classification) from HF Hub on the co-registered Palisades imagery, producing a 3-class damage map (background, intact, damaged) matching the xBD-S12 label schema.
7. **Output Validation:** Verified the damage map by overlaying it on post-event optical imagery to confirm correspondence with the visible burn scar; computed summary stats ($\approx 4.2\%$) of scene classified as damaged buildings, spatially concentrated rather than scattered—yielding a validated, damage-labeled S1/S2 pair for the Palisades event.

Work Done by Ridita Afrin Riya

1. **System Architecture and Pipeline Development:** Designed and implemented “DisasTeller,” an autonomous, multi-agent AI framework utilizing CrewAI, LangChain, and Large Vision-Language Models (Google Gemini 3.1 Flash-Lite). The system was engineered to simulate a real-world emergency command center, specifically targeting and resolving critical information bottlenecks in post-disaster scenarios through instant visual data processing.
2. **Specialized Multi-Agent Orchestration:** Replaced the traditional monolithic AI approach by dividing complex disaster response workloads across four specialized agents. Configured the *Expert Team* for visual damage grading from disaster photos, the *Alerts Team* for issuing public hazard warnings, the *Emergency Team* for staging triage shelters, and the *Assignment Team* for managing human resource logistics and budgets through a sequential chain of command.
3. **API Mitigation and System Stabilization:** Addressed critical system crashes caused by multi-agent parallel querying that rapidly exhausted Gemini’s free-tier limits (resulting in 429 RESOURCE_EXHAUSTED errors). Successfully mitigated these issues by migrating to the highly efficient `gemini-3.1-flash-lite` model and implementing a strict `max_rpm=2` constraint within CrewAI to act as a pacing mechanism, alongside resolving dynamic environment pathing for stable deployment.
4. **Simulation Testing and Validation:** Executed and validated a full-scale simulation based on the Wajima Earthquake scenario. Demonstrated the system’s ability to autonomously analyze field images, correctly identify a severed bridge as a “Grade 5 Total Collapse,” systematically route 80 Search and Rescue personnel to the exact location, and strategically coordinate the setup of a nearby medical triage center.

***Next Week’s Plan:** Complete the remaining training steps (102/217→217/217) and execute the vLLM evaluation pipeline against the 6 MCQ benchmark tasks from Table 2. Report accuracy metrics (DSR, DTR, BBR, BDC, DRE, ORR) and compare against the paper’s published baselines to quantify the impact of our hardware-constrained QLoRA adaptation.